

TokenPack: Modeling LLM Context Window Selection as a 0/1 Knapsack Problem

Metehan Kizilcik

Department of Software Engineering

HFTTF

Email: 232803055@ogr.cbu.edu.tr

Abstract—Large language models cannot always consume full long documents because context windows are limited and expensive. This paper models retrieval-time context selection as a 0/1 Knapsack Problem in which each semantic chunk is an item, token count is weight, and query-conditioned salience is value. TokenPack compares naive ranking, budget-aware retrieval heuristics, and knapsack selection under the same token budget across synthetic optimization tests, QASPER evidence benchmarks, LLMingua-style compression comparisons, and a LongBench v2 generation pilot. Exact dynamic programming gives the optimum for manageable instances but becomes impractical as the nC state space grows, while density-aware and redundancy-aware selection remain efficient. On QASPER, the strongest result is not universal selector dominance, but a systems finding: at approximately matched 50% token savings, structure-aware TokenPack selection preserves substantially more evidence than token-level LLMingua-2 compression alone. On LongBench v2, TokenPack remains close to full-context Qwen2.5-14B accuracy at roughly 50% savings, while the clearest positive signal is a TokenPack-then-LongLLMingua cascade that reaches 74.5% average context reduction with near-full-context accuracy.

Index Terms—Knapsack, 0/1 Knapsack, Greedy Algorithm, Dynamic Programming, LLM, RAG, Context Window, Algorithm Analysis

I. INTRODUCTION

NP-Hard problems such as the Traveling Salesman Problem, Warehouse Location Problem, and Knapsack Problem are commonly used in algorithm analysis to compare exact and heuristic algorithms. This study applies the 0/1 Knapsack Problem to a modern setting: context window selection for large language models (LLMs).

When a long document is divided into candidate chunks, only a subset can be placed into the limited token budget of an LLM. This selection task naturally follows a knapsack structure. TokenPack models the context window as a capacity, text chunks as items, token counts as weights, and query relevance scores as values.

The central research question is: can a classical NP-Hard optimization model improve LLM context selection compared with retrieval heuristics that only rank chunks independently? This question is important because LLM context windows are expensive and finite resources. A chunk with high semantic relevance but very large token cost may not be the best choice if several shorter chunks collectively provide more useful evidence. Therefore, the problem is not only a retrieval problem; it is a constrained optimization problem.

The contribution of this work is fourfold. First, it presents a direct mapping between LLM context selection and the 0/1 Knapsack Problem. Second, it implements exact and heuristic algorithms in a reusable Python package and command-line tool. Third, it evaluates chunking, scoring, and selection decisions on both controlled synthetic instances and QASPER evidence annotations. Fourth, it adds direct comparisons between retrieval-time selection, prompt-compression baselines, and long-context generation accuracy, showing where selection-first and compression-first pipelines differ.

II. LITERATURE REVIEW

The Knapsack Problem is one of the fundamental NP-Hard problems in combinatorial optimization [1], [2]. Many extensions and solution strategies have been studied, including multidimensional, multiobjective, and multiple-choice variants [3]–[5]. Metaheuristic optimization methods such as NSGA-II are also frequently used for combinatorial optimization when exact methods are too expensive [6].

Retrieval-Augmented Generation (RAG) provides external knowledge to LLMs by retrieving relevant document chunks [7]. However, simple top-k retrieval does not directly optimize token budget usage. Long-context methods such as MemGPT, Parallel Context Windows, LongRoPE, PoSE, and positional interpolation show that context length is a major practical limitation in LLM systems [8]–[12]. Recent evaluations also compare long-context models and RAG, and show that effective context length and computational cost still matter even when nominal context windows grow [13]–[16].

Chunking and embedding quality are also central to retrieval. Late Chunking, S2 Chunking, Landmark Embedding, and document chunking studies demonstrate that segmentation and representation choices affect downstream retrieval quality [17]–[21]. Context pruning and query-aware sparsity methods such as AttentionRAG, Quest, SnapKV, WindowKV, and BudgetMem further support the idea that not every token should be placed into the model context [22]–[26]. Prompt-compression methods such as LLMingua, LongLLMingua, and LLMingua-2 attack the same budget problem from a different angle: instead of selecting chunks before prompting, they compress the prompt tokens after context construction [27]–[29]. BudgetMem is especially relevant because it keeps selection at the chunk level, uses BM25 retrieval after ingestion-time filtering, and scores chunks with interpretable signals such as TF-IDF, position

bias, entity density, numerical density, and discourse markers. These neighboring lines of work motivate a broader TokenPack question: the optimizer should be tested not only with different solvers, but also against compression-first alternatives that operate at a different stage of the pipeline.

Knapsack-style formulations have also appeared in summarization and recent LLM optimization work. Tree knapsack summarization and budget-aware summarization explicitly connect content selection with budget constraints [30], [31]. Recent LLM-oriented work uses knapsack-like ideas for NP-Hard problem evaluation, exploration budget allocation, and speculative decoding [32]–[34]. TokenPack follows this direction but focuses specifically on retrieval-time context packing.

III. PROBLEM FORMULATION

Given a query q , a document is split into n chunks. Each chunk i has a value v_i and a weight w_i . In the default TokenPack setting, v_i is computed from cosine similarity between the query embedding and chunk embedding, while w_i is the chunk token count. TokenPack also includes a hybrid scoring profile inspired by practical RAG and selective-memory systems:

$$v_i = 0.55c_i + 0.25b_i + 0.15p_i + 0.05h_i \quad (1)$$

where c_i is normalized embedding cosine similarity, b_i is normalized BM25 lexical relevance, p_i is a U-shaped position prior that favors beginning and ending chunks, and h_i is adjacent-chunk coherence. These coefficients are hand-set rather than learned: they intentionally keep semantic similarity dominant, give lexical matching the second-largest role, and treat position and coherence as smaller stabilizing priors. They should therefore be read as a transparent heuristic baseline, not as a tuned optimum. The effective LLM context budget is denoted by C .

Later ablations test two richer hand-designed profiles. The *evidence-hybrid* profile reallocates weight away from position and toward explicit query-term coverage and structural priors such as titles, headers, and list/table cues, using weights 0.35/0.25/0.20/0.15/0.05 over cosine, BM25, query coverage, structural prior, and neighbor coherence. The *knapsack-aware* profile then treats evidence-hybrid as a base score and blends it with value-density, length-utility, and term-specificity priors using weights 0.70/0.15/0.10/0.05. These profiles are included to test score alignment, not to claim that the coefficients themselves are theoretically optimal; no hyperparameter sensitivity sweep is claimed in this version. For reproducibility, the reported QASPER and Long-Bench runs use the deterministic hashing-384 embedding backend unless otherwise stated; the package also supports sentence-transformers/all-MiniLM-L6-v2 as the default neural embedding model when the sentence-transformers backend is selected.

The optimization problem is:

$$\max \sum_{i=1}^n v_i x_i \quad (2)$$

$$\sum_{i=1}^n w_i x_i \leq C, \quad x_i \in \{0, 1\} \quad (3)$$

Here, $x_i = 1$ means that chunk i is selected. The objective maximizes total semantic utility while preserving the hard token budget. This formulation is more suitable than top-k retrieval when chunk lengths are unequal, because top-k ignores item weight.

IV. METHOD

The TokenPack pipeline has four main stages:

- 1) The long document is split into chunks using paragraph-group or semantic-threshold chunking.
- 2) Each chunk and query is embedded into a vector space.
- 3) Chunk value is computed using either cosine similarity or the hybrid salience score, and chunk weight is computed using token count.
- 4) Selection is performed with top-k, budget-top-k, MMR, greedy heuristics, or 0/1 knapsack dynamic programming.

A. Semantic Chunking

Chunking is the stage that converts an unstructured long document into knapsack items. This is important because the optimization algorithm can only choose among the chunks that the preprocessing stage creates. If chunk boundaries are poor, even an optimal knapsack solver may select incomplete or semantically broken evidence.

TokenPack supports two chunking modes. The first mode is **paragraph-group chunking**. The document is first split into structural blocks such as paragraphs or PDF text blocks. Consecutive blocks are then grouped until the chunk reaches a target token range. In this study, the default target is 650 tokens with 120–900 token bounds, chosen as an engineering compromise between preserving local context and giving the optimizer enough items to pack under tight budgets. The chunking ablation varies the boundary construction method, but it does not independently sweep chunk-size ranges. Each chunk keeps metadata such as source path, page number when available, block order, and character offsets. This metadata is important because selected chunks are later restored to their original document order before being sent to the LLM.

The second mode is **semantic-threshold chunking**. Instead of grouping only by length, this mode compares neighboring text blocks in embedding space. For two adjacent blocks b_i and b_{i+1} , TokenPack computes:

$$s_i = \cos(E(b_i), E(b_{i+1})) \quad (4)$$

where $E(\cdot)$ is the embedding function. If s_i falls below a threshold, the algorithm treats this point as a topic shift and starts a new chunk. A maximum token limit is still enforced, so semantic chunking cannot create chunks that violate the

later token-budget assumptions. This makes the chunker both semantic and budget-aware.

The purpose of semantic chunking is not to solve the optimization problem by itself. Instead, it improves the quality of the item set given to knapsack. Paragraph grouping provides stable and explainable boundaries, while semantic-threshold chunking can split documents at topic changes. In both cases, the output item must contain three values needed by the optimizer: text content, token weight, and source-order metadata.

B. Compared Algorithms

Dynamic Programming (DP). DP computes the optimal subset under the token capacity. The implementation uses a one-dimensional value table for the objective and a reconstruction table for selected weight. Its time complexity is $O(nC)$.

Greedy by Value. This heuristic sorts chunks by value and inserts each chunk if it fits. It is fast, but it may choose large chunks that block better combinations.

Greedy by Value Density. This heuristic sorts chunks by v_i/w_i . It is usually stronger than value-only greedy because it accounts for token efficiency, but it is still not guaranteed to find the optimal 0/1 solution.

Greedy by Lightest Weight. This baseline chooses short chunks first. It tests whether token minimization alone is enough. In practice, it ignores semantic value and therefore may select weak evidence.

Random Feasible. This baseline shuffles chunks and inserts feasible items. It gives a lower-bound style comparison for non-optimized selection.

Simulated Annealing (SA). SA starts from a density-greedy feasible solution and then explores neighboring subsets by adding or removing one item at a time. It uses temperature $\max(1, n/2)$, $\min(25000, \max(2000, 12n))$ iterations, multiplicative cooling by 0.9995, and a 1.02 deterministic reheating every 250 steps. This provides a metaheuristic baseline that is more exploratory than simple greedy selection.

Document-prefix and Full-document Baselines. These two baselines represent the system without TokenPack-style optimization. Document-prefix inserts chunks in the original PDF order until the budget is full. Full-document tries to insert the entire source document. These baselines are useful because they show what happens when semantic value and token cost are not optimized jointly.

Top-k, Budget-top-k, and MMR. Top-k selects a fixed number of chunks only by similarity. Budget-top-k adds a feasibility check. MMR additionally penalizes redundancy between selected chunks. The `knapsack-redundancy` strategy keeps MMR as a separate algorithm, but applies a redundancy penalty to chunk values before the knapsack solver runs. This is a practical linear approximation rather than a full quadratic knapsack formulation.

These algorithms were not only described theoretically; they were executed on the same generated instances so that their differences can be measured statistically. Table I reports a paired comparison for $N = 1000$ over 100 runs. The word

Table I
PAIRED STATISTICAL COMPARISON OF ALGORITHMS FOR $N = 1000$ OVER 100 RUNS

Algorithm	Mean Value	Mean Time (ms)	Gap (%)	95% CI Gap	Speedup
DP exact	111281.2	56.644	0.00	0.00	–
Greedy density	111252.4	0.209	0.03	0.01	271.3x
Simulated annealing	111252.4	3.784	0.03	0.01	15.0x
Greedy lightest	95924.7	0.150	13.84	0.38	376.4x
Greedy value	40785.1	0.147	63.30	0.65	386.4x
Random feasible	20897.2	0.173	81.20	0.43	326.8x

“paired” means that every algorithm receives the same item values, weights, and capacity for each repetition. Therefore, the measured gap is caused by the algorithmic strategy rather than by different random inputs. In the tables, shaded rows mark the TokenPack-relevant exact knapsack baseline and its practical density-based heuristic.

The comparison shows a clear trade-off. DP is the exact baseline and has zero gap, but it is much slower. Greedy density is approximately 271 times faster than DP in this setting while keeping the mean gap at only 0.03%. Simulated annealing reaches a similar gap in this experiment, but it is slower than density-greedy because it performs additional neighborhood exploration. Greedy value is also fast, but its mean gap is 63.30%, which shows that choosing high-value items without considering weight is unreliable. The lightest-first baseline is faster than DP but loses 13.84% value on average, proving that minimizing tokens alone is also not enough.

C. Why Knapsack is Appropriate

The mapping from context selection to knapsack is direct. The context window is the capacity, chunks are items, token counts are weights, and query relevance scores are values. This model is more faithful than top-k because it optimizes total information value under an explicit constraint. It also supports fair algorithm analysis: exact DP can be compared with greedy heuristics using runtime, timeout rate, and accuracy gap.

V. COMPLEXITY ANALYSIS

For n candidate chunks and effective token budget C , the DP algorithm has $O(nC)$ time complexity. This is pseudo-polynomial: it is efficient for moderate token budgets but can become expensive when either the number of chunks or the capacity is large. Greedy algorithms require sorting, so their time complexity is $O(n \log n)$ and memory usage is low.

This theoretical expectation leads to a practical hybrid design. Embedding similarity first prunes the candidate pool, then a knapsack optimizer is applied only to the strongest candidates. This keeps DP feasible while preserving the main advantage of exact budget-aware optimization.

VI. EXPERIMENTAL DESIGN

Nine experiments are used.

Repeated synthetic knapsack benchmark. Random instances are generated for $N \in \{100, 500, 1000, 5000, 10000\}$. Each setting is repeated 100 times with different random seeds. DP is treated as the exact baseline when $nC \leq 5,000,000$ states. For larger instances, DP is marked as timeout to represent the

practical point where the exact method becomes too expensive. The reported metrics are mean objective value, mean runtime, standard deviation of runtime, mean accuracy gap from DP optimum, maximum gap, and timeout rate.

Retrieval feasibility benchmark. A small human-reviewed gold-evidence benchmark checks whether retrieval strategies obey the token budget. This benchmark compares no-optimization baselines, top-k, budget-top-k, MMR, and knapsack on selected document chunks. Its purpose is not to prove final QA quality, but to verify the main engineering constraint: a context selection algorithm must not exceed the budget.

Chunking ablation benchmark. A sample of PDFs from the `resources` directory is converted into sentence-level source blocks and indexed three ways: paragraph-group, semantic-threshold, and structure-aware chunking. The evidence targets are defined before chunking as source text spans represented by content terms, so the evaluation does not favor either chunker by using chunk IDs from one index. The solver is fixed to knapsack, scoring is fixed to the evidence-hybrid profile, and budgets are set as 1%, 3%, and 5% of source tokens. This isolates the effect of chunk boundaries from the effect of the selection algorithm and the value function.

Value-function ablation benchmark. The same PDF sample and evidence-term targets are also used to compare several scoring profiles, from cosine-only similarity to richer hybrid and evidence-aware variants. In this experiment, chunking is fixed to structure-aware and selection is fixed to knapsack. Therefore, the only changed variable is the value function used by the optimizer.

QASPER selector ablation. To avoid relying only on the small manually reviewed PDF benchmark, TokenPack is also evaluated on QASPER validation examples with evidence annotations. The compared selectors are budget-top-k, greedy-density, knapsack, and knapsack-redundancy. Chunking is fixed to structure-aware and scoring is fixed to the evidence-hybrid value function, so the experiment asks whether the selector itself changes evidence coverage under the same chunk values. Budgets are defined as 20%, 30%, 40%, and 50% of each paper’s source tokens.

QASPER cost-quality proxy. The main practical goal is to reduce input-token cost while preserving answer quality. Because no local or cloud LLM is assumed to be available for every run, this experiment measures answerability proxies on QASPER. For each selector and budget, it records selected-token cost, retained gold-answer token coverage, retained evidence coverage, and the rate at which both answer and evidence are preserved. The answer-token-overlap F1 metric is computed between the selected context text and the gold answer string after lowercasing and token matching. It is therefore a retention proxy rather than a model-output quality score, and its absolute values are expected to be low because the retained context is much longer than the short gold answer and some QASPER answers are mildly abstractive. Scores are reported relative to a 100% high-budget RAG setting using the same chunker, scoring function, and selector.

QASPER compression comparison. Selection and com-

pression solve related but distinct problems, so they are compared directly on the same QASPER questions. The 200-question subset is not sampled randomly: the script streams QASPER validation papers in dataset order and stops after the first 200 questions from papers with parseable blocks and questions. Three pipelines are evaluated: Only TokenPack, Only LLMingua-2, and TokenPack followed by LLMingua-2. The metrics are final token saving, evidence recall, complete-evidence rate, answer-token-overlap F1, latency, and per-question win rates.

LongBench v2 generation pilot. To test whether the evidence-retention signal transfers to downstream long-context answering, a separate Modal-based experiment evaluates the eligible LongBench v2 multiple-choice cases [35]. The generator is Qwen2.5-14B-Instruct [36]. Source contexts are filtered to the 8k–24k token range so that full-context prompting remains feasible within the configured 32k model length. The released split used for this pilot contains 503 examples; the script streams examples in dataset order and yields 83 eligible cases after skipping 420 examples above 24k source tokens and none below 8k. Four methods are compared: full context, TokenPack at a 50% source-token budget with score-then-source context ordering, LongLLMingua at rate 0.50, and TokenPack followed by LongLLMingua at rate 0.50. The LongLLMingua baseline is run in context-level filtering mode, with token-level filtering disabled, so it should be interpreted as a practical LongLLMingua-style context-pruning baseline rather than a full token-level compression study. Accuracy is exact multiple-choice correctness after parsing the model’s A/B/C/D answer.

Illustrative input-cost estimate. A final accounting table scales the LongBench v2 context-reduction ratios to a 1M baseline-token scenario using public API pricing assumptions [37]. This estimate is included only to show the economic implication of token reduction; it is not used as evidence for answer quality.

Reproducibility. The implementation is included in the accompanying artifact as the `tokenpack` Python package, with a source manifest listing modules, tests, Modal commands, and QASPER reproduction scripts. Until a public repository URL is attached, the package claim should be interpreted as artifact-based rather than independently archival.

VII. EXPERIMENTAL RESULTS

Figure 1 summarizes the selection-first pipeline and compression frontier, while Figure 2 visualizes runtime behavior. The y-axis is logarithmic because DP grows much faster than greedy methods, matching the theoretical difference between $O(nC)$ dynamic programming and $O(n \log n)$ greedy sorting.

Figure 3 shows the approximation gap for $N = 1000$. Greedy density is visually almost indistinguishable from DP in terms of objective value, while greedy value and random feasible selection lose a large portion of the optimum. This is the most important quality comparison: not every fast heuristic is equally reliable.

Selection-first context pipeline



Token budget is enforced before optional prompt compression.

QASPER compression frontier

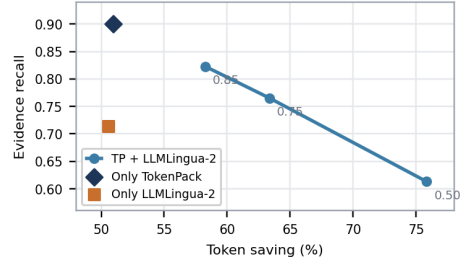


Figure 1. Visual summary of the selection-first pipeline and QASPER compression frontier from Tables IX and X.

Runtime Scaling Across Algorithms

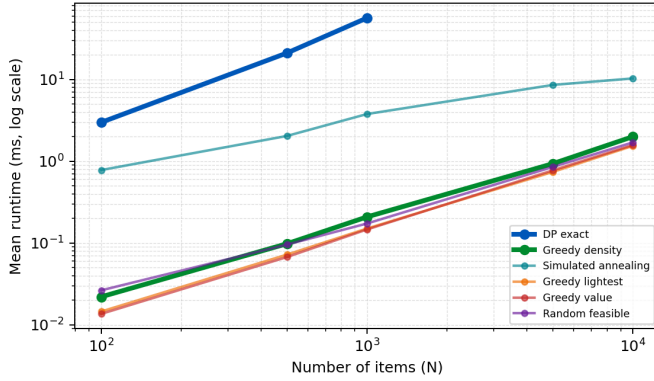


Figure 2. Runtime scaling of exact and heuristic algorithms across problem sizes.

Approximation Gap at $N=1000$

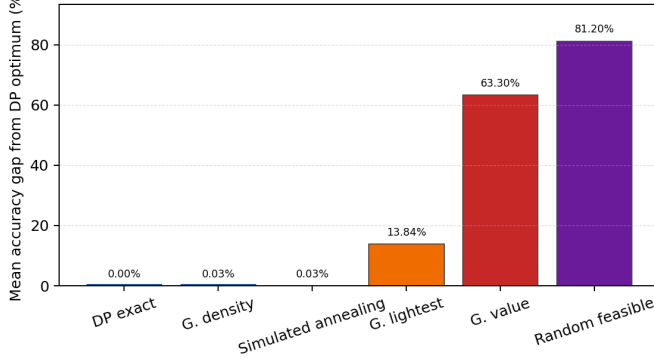


Figure 3. Mean approximation gap from the DP optimum for $N = 1000$ over 100 paired runs.

Table II reports the same repeated synthetic results numerically. For $N = 100$ and $N = 1000$, DP gives the optimum but is slower than greedy methods. At $N = 1000$, DP takes 56.644 ms on average, while greedy density takes only 0.209 ms. However, greedy density remains very close to optimum, with only 0.03% mean gap. Greedy value is much weaker: its mean gap reaches 63.30% at $N = 1000$.

Figure 4 shows why DP cannot be used blindly on very large candidate pools. The x-axis is the DP state space nC . The red timeout points appear after the configured state threshold.

Table II

REPEATED SYNTHETIC KNAPSACK RESULTS (100 RUNS PER SIZE)

N	Algorithm	Mean Value	Mean Time (ms)	Mean Gap (%)	Timeout Rate
100	DP exact	35491.8	2.991	0.00	0.00
100	Greedy density	35439.0	0.022	0.15	0.00
100	Greedy value	31962.1	0.014	9.95	0.00
100	Random feasible	20124.2	0.026	43.35	0.00
100	Simulated annealing	35439.8	0.781	0.15	0.00
1000	DP exact	111281.2	56.644	0.00	0.00
1000	Greedy density	111252.4	0.209	0.03	0.00
1000	Greedy value	40785.1	0.147	63.30	0.00
1000	Random feasible	20897.2	0.173	81.20	0.00
1000	Simulated annealing	111252.4	3.784	0.03	0.00
10000	DP exact	—	—	—	1.00
10000	Greedy density	342178.6	2.001	—	0.00
10000	Greedy value	43537.4	1.582	—	0.00
10000	Random feasible	21552.6	1.701	—	0.00
10000	Simulated annealing	342178.6	10.320	—	0.00

Dynamic Programming Scalability

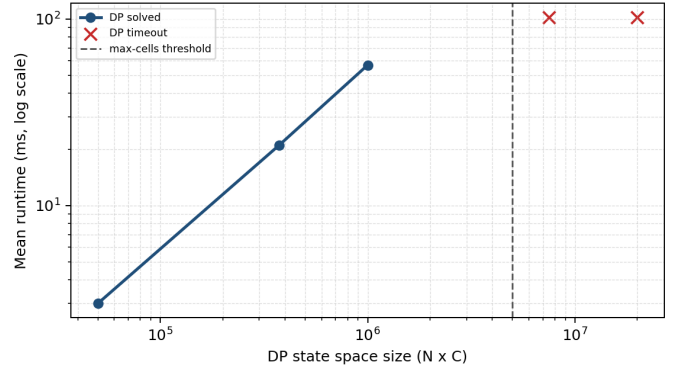


Figure 4. Dynamic programming runtime and timeout behavior as the nC state space grows.

This directly illustrates the pseudo-polynomial limitation of the exact method.

Table III gives the exact state counts behind Figure 4. DP works for 50,000, 375,000, and 1,000,000 states in this experiment, but it is skipped at 7,500,000 and 20,000,000 states. This directly demonstrates the pseudo-polynomial limitation of dynamic programming.

Table III
DP SCALABILITY AS nC GROWS

N	Capacity	nC States	Timeout Rate
100	500	50000	0.00
500	750	375000	0.00
1000	1000	1000000	0.00
5000	1500	7500000	1.00
10000	2000	20000000	1.00

Table IV
RETRIEVAL FEASIBILITY WITH AND WITHOUT TOKENPACK

Eff. Budget	Method	Recall	Avg. Tokens	Over Budget
250	document-prefix	0.25	180	0.00
250	full-document	1.00	1539	1.00
250	top-k	0.94	809	1.00
250	TokenPack knapsack	0.75	213	0.00
700	document-prefix	0.69	698	0.00
700	full-document	1.00	1539	1.00
700	top-k	0.94	809	0.88
700	TokenPack knapsack	0.94	657	0.00

Table IV shows why TokenPack improves the process rather than only adding implementation complexity. Without TokenPack, the document-prefix baseline is budget-valid but misses much of the evidence. The full-document baseline reaches perfect recall only by violating the budget. Plain top-k is semantically strong but also frequently over budget. Knapsack selection gives the desired middle point: high evidence recall while keeping the selected context inside the effective token capacity.

Table V
CHUNKING ABLATION WITH THE SAME KNAPSACK SOLVER

Chunker	Budget	Chunks	Avg. Tok.	Term Recall	Complete	Avg. Used	Util.
paragraph	1%	639	259.800	0.979	0.958	1653.833	0.997
paragraph	3%	639	259.800	1.000	1.000	4976.417	0.999
paragraph	5%	639	259.800	1.000	1.000	8296.583	1.000
semantic-threshold	1%	1332	124.610	0.798	0.583	1658.667	1.000
semantic-threshold	3%	1332	124.610	0.854	0.625	4978.833	1.000
semantic-threshold	5%	1332	124.610	0.894	0.750	8278.625	0.998
structure-aware	1%	639	259.800	0.979	0.958	1653.833	0.997
structure-aware	3%	639	259.800	1.000	1.000	4976.417	0.999
structure-aware	5%	639	259.800	1.000	1.000	8296.583	1.000

Table V adds the missing chunking ablation. Because the solver and scoring profile are fixed, differences come from the item set created before optimization. On this PDF sample, paragraph-group and structure-aware chunking behave almost identically and both dominate semantic-threshold chunking at every tested budget. At the tight 1% budget, paragraph-group and structure-aware reach 0.979 evidence-term recall, while semantic-threshold reaches 0.798. By 3%, the paragraph-group and structure-aware variants already recover all evidence terms, whereas semantic-threshold remains below that level even at 5%. In this dataset, over-segmentation appears to fragment supporting evidence rather than help the optimizer use budget more effectively.

Table VI isolates the value function. With the same structure-aware chunks and the same knapsack solver, score design matters much more than the choice between exact and heuristic selection. Cosine alone reaches only 0.410 evidence-term recall and 0.083 complete-evidence rate at the tight 1% budget.

Table VI
VALUE FUNCTION ABLATION WITH FIXED CHUNKING AND KNAPSACK SOLVER

Scoring	Budget	Chunks	Avg. Tok.	Term Recall	Complete	Avg. Used	Redund.
cosine	1%	639	259.800	0.410	0.083	1652.750	0.021
cosine	3%	639	259.800	0.811	0.625	4975.583	0.020
cosine	5%	639	259.800	0.923	0.833	8296.958	0.021
hybrid	1%	639	259.800	0.588	0.333	1654.792	0.022
hybrid	3%	639	259.800	1.000	1.000	4975.500	0.020
hybrid	5%	639	259.800	1.000	1.000	8295.833	0.021
evidence-hybrid	1%	639	259.800	0.979	0.958	1653.833	0.025
evidence-hybrid	3%	639	259.800	1.000	1.000	4976.417	0.026
evidence-hybrid	5%	639	259.800	1.000	1.000	8296.583	0.028
knapsack-aware	1%	639	259.800	0.826	0.708	1654.833	0.023
knapsack-aware	3%	639	259.800	1.000	1.000	4975.458	0.021
knapsack-aware	5%	639	259.800	1.000	1.000	8296.125	0.023
instruction-ami	1%	639	259.800	0.979	0.958	1653.833	0.025
instruction-ami	3%	639	259.800	1.000	1.000	4976.417	0.026
instruction-ami	5%	639	259.800	1.000	1.000	8296.583	0.028

Richer profiles improve sharply: the plain hybrid score reaches 0.588 recall, knapsack-aware reaches 0.826, and evidence-hybrid reaches 0.979 with a 0.958 complete-evidence rate. The *instruction-ami* row is an experimental AMI hook; because no local AMI model is configured in this run, it falls back to evidence-hybrid and therefore matches it exactly. By 3%, several richer profiles already recover all evidence terms.

Table VII
QASPER SELECTOR EVALUATION WITH FIXED SCORING

Selector	Budget	Avg Tok.	Evidence Recall	Complete	Answer Overlap F1	Value	Lat.
budget-top-k	20%	982.549	0.713	0.424	0.028	2.257	0.000
greedy-density	20%	982.549	0.694	0.382	0.028	2.339	0.000
knapsack	20%	982.549	0.703	0.401	0.027	2.383	0.002
knapsack-redundancy	20%	982.549	0.707	0.407	0.027	2.105	0.009
budget-top-k	30%	1473.954	0.815	0.596	0.021	3.174	0.000
greedy-density	30%	1473.954	0.804	0.570	0.021	3.267	0.000
knapsack	30%	1473.954	0.807	0.571	0.020	3.313	0.003
knapsack-redundancy	30%	1473.954	0.807	0.571	0.020	2.883	0.010
budget-top-k	40%	1965.483	0.872	0.718	0.017	3.991	0.000
greedy-density	40%	1965.483	0.869	0.704	0.017	4.080	0.000
knapsack	40%	1965.483	0.871	0.708	0.016	4.123	0.005
knapsack-redundancy	40%	1965.483	0.871	0.711	0.016	3.555	0.012
budget-top-k	50%	2457.100	0.911	0.818	0.014	4.722	0.000
greedy-density	50%	2457.100	0.908	0.799	0.014	4.795	0.000
knapsack	50%	2457.100	0.908	0.799	0.014	4.837	0.006
knapsack-redundancy	50%	2457.100	0.906	0.793	0.014	4.146	0.013

Table VII evaluates the central selector question on 861 QASPER validation questions from 276 papers. The scoring function and chunker are fixed to the strongest tested configuration, so differences come from the selection rule rather than from chunk boundaries. The results are important precisely because they are not a clean win for knapsack: budget-top-k gives the highest evidence recall at every tested budget, while knapsack and knapsack-redundancy are close but slightly lower. At the same time, the knapsack variants remain competitive and are usually above or near density-greedy. This suggests that the main bottleneck is not the solver itself but the utility model being optimized. In other words, exact or near-exact budgeted optimization is only as good as the scoring function it receives.

Table VIII evaluates the practical cost-quality trade-off more directly. The 100% budget row is treated as the high-budget RAG reference. Under the stronger structure-aware and evidence-hybrid configuration, both budget-top-k and TokenPack preserve most of the answerability proxy at moderate budgets, and their frontier is tighter than the earlier semantic-threshold runs suggested. At 80% budget, both retain about 97.5–97.8% of evidence recall while saving roughly 21–22% of tokens. At 70% budget, both remain around 96.0% evidence

Table VIII
QASPER COST-QUALITY PROXY UNDER REDUCED CONTEXT BUDGETS

Selector	Budget	Used Tok.	Saving	Ans. vs 100	Ev. vs 100	Preserved
<i>budget-top-k</i>	50%	2405	51.4%	0.921	0.902	0.759
TokenPack knapsack-redundancy	50%	2429	50.9%	0.914	0.896	0.735
<i>budget-top-k</i>	70%	3382	31.5%	0.967	0.960	0.878
TokenPack knapsack-redundancy	70%	3403	31.1%	0.971	0.960	0.892
<i>budget-top-k</i>	80%	3864	21.8%	0.978	0.975	0.926
TokenPack knapsack-redundancy	80%	3883	21.4%	0.981	0.978	0.925

Table IX
QASPER COMPRESSION COMPARISON AT APPROXIMATELY MATCHED 50% TOKEN SAVING

Method	Saving	Evidence Recall	Complete	Answer Overlap F1	Sec.
Only LLMingua-2 rate=0.50	50.6%	0.714	0.110	0.010	6.28
Only TokenPack	50.9%	0.900	0.785	0.013	0.01
TokenPack + LLMingua-2 rate=0.50	75.8%	0.613	0.045	0.017	3.37

Table X
COMPRESSION FRONTIER ON QASPER (ALL ROWS USE 200 QUESTIONS)

Method	Saving	Evidence Recall	Answer Overlap F1	Run
Only LLMingua-2 rate=0.50	50.6%	0.714	0.010	current-200q
Only TokenPack	50.9%	0.900	0.013	current-200q
TokenPack + LLMingua-2 rate=0.85	58.3%	0.823	0.014	current-200q
TokenPack + LLMingua-2 rate=0.75	63.4%	0.765	0.014	current-200q
TokenPack + LLMingua-2 rate=0.50	75.8%	0.613	0.017	current-200q

Run notes: all rows are refreshed *current-200q* runs on the same 200-question split.

retention with roughly 31% token saving. At 50% budget, evidence retention falls to about 0.90 for budget-top-k and 0.896 for knapsack-redundancy. The practical lesson is again conservative: selector optimization helps, but under the current scoring function the best retrieval-time baseline is still budget-top-k, and more aggressive savings require either stronger value estimation or a second-stage compressor.

Table IX compares retrieval-time selection with token-level prompt compression on the same 200 QASPER validation questions at approximately matched 50% token savings. Only TokenPack saves 50.9% of input tokens while preserving 0.900 evidence recall. Only LLMingua-2 reaches a nearly identical 50.6% saving, but its evidence recall drops to 0.714. The complete-evidence rate gap is even larger: 0.785 for TokenPack versus 0.110 for LLMingua-2. A per-question analysis shows that TokenPack achieves higher evidence recall than LLMingua-2 on 84.0% of the evaluated questions. The answer-token-overlap F1 values stay numerically small because they measure answer-string retention inside the selected context, not generated answer correctness. This is the clearest empirical result of the paper: when the final budget is matched, structure-aware evidence selection preserves answer-bearing spans better than token-level compression alone.

Table X extends the comparison to stronger second-stage compression. All rows are refreshed on the same 200-question QASPER validation split with structure-aware chunks, evidence-hybrid scoring, knapsack-redundancy selection, and LLMingua-2 as the second-stage compressor. TokenPack followed by LLMingua-2 at rate 0.85 preserves 0.823 evidence recall while reaching 58.3% total token saving, which is better than Only LLMingua-2 at 50.6% saving and 0.714 evidence recall. More aggressive second-stage compression continues to increase savings, but the frontier shows a consistent pattern: the

first quality-preserving step is retrieval-time evidence selection, while prompt compression is best interpreted as an optional second-stage cost reducer with a visible recall trade-off.

Table XI
LONGBENCH V2 END-TO-END MULTIPLE-CHOICE ACCURACY. TP DENOTES TOKENPACK; LLL DENOTES LONGLLMLINGUA CONTEXT-LEVEL FILTERING.

Method	Runs	Acc.	Avg. Ctx.	Saving
Full context	83	0.398	17646	0.000
TP-50	83	0.386	8758	0.504
LLL-0.50	83	0.402	8560	0.512
TP-50 + LLL-0.50	83	0.410	4486	0.745

Table XI adds an end-to-end long-context generation view. Unlike the QASPER compression tables, this experiment measures final multiple-choice correctness rather than evidence-retention proxies. On 83 eligible LongBench v2 cases, full-context prompting reaches 0.398 accuracy, TokenPack reaches 0.386 at roughly half the input size, and the context-level LongLLMLingua baseline is slightly higher at 0.402. These 0.386–0.410 differences are small relative to the roughly ± 0.11 binomial uncertainty of an 83-case pilot, so the table should not be read as a statistically significant win for any single method. The clearer systems signal is the cascade: TokenPack followed by LongLLMLingua reaches 74.5% saving while retaining 0.410 accuracy.

Table XII
LONGBENCH V2 HOT-MODEL LATENCY RERUN. TP DENOTES TOKENPACK; LLL DENOTES LONGLLMLINGUA CONTEXT-LEVEL FILTERING. LATENCY IS MEASURED WITH BATCH SIZE ONE AND EXCLUDES MODAL COLD START/MODEL LOADING.

Method	Runs	Saving	Prep s	LLM s	Total s	Speedup
Full context	83	0.000	0.000	4.106	4.106	1.00×
TP-50	83	0.504	0.019	2.166	2.184	1.88×
LLL-0.50	83	0.512	0.537	1.895	2.432	1.69×
TP-50 + LLL-0.50	83	0.745	0.289	0.835	1.124	3.65×

Table XII reruns the same 83-case LongBench v2 setting with batch size one to measure hot-model wall-clock latency per request. The measurement excludes Modal cold start and model loading, but includes the TokenPack selection and LongLLMLingua preprocessing time that occurs before the answer request. TokenPack alone reduces mean total latency from 4.106s to 2.184s, a 1.88× speedup. The TokenPack-then-LongLLMLingua cascade reduces the final context to 4,486 tokens on average and reaches 1.124s mean total latency, a 3.65× speedup over full-context prompting.

Table XIII and Figure 5 show the economic implication of the LongBench v2 context-reduction ratios. TokenPack alone reduces the scaled input from 1,000,000 to roughly 496,000 paid tokens, while the TokenPack-then-LongLLMLingua cascade reduces it to roughly 255,000 paid tokens, corresponding to the 74.5% average saving reported in Table XI. The exact dollar value changes with provider pricing, but the token reduction ratio is determined by the selection and compression strategy.

Table XIII
ESTIMATED INPUT COST SAVINGS FROM LONGBENCH v2 CONTEXT
REDUCTION

Method	Paid Tokens	Cost (\$)	Saving vs Full Doc
Full context	1000000	2.500	0.0%
TP-50	495725	1.239	50.4%
LLL-0.50	487808	1.220	51.2%
TP-50 + LLL-0.50	255218	0.638	74.5%

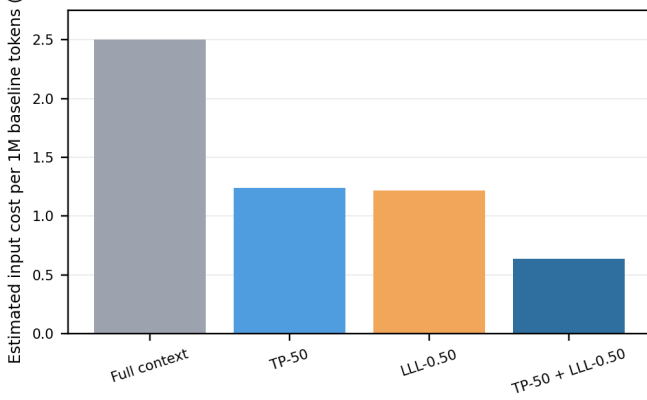


Figure 5. Estimated input-token cost under a 1M baseline-token scenario.

VIII. DISCUSSION

The experiments support six observations. First, exact DP is the most reliable method when the instance size is manageable because it gives the optimum. Second, the $O(nC)$ cost becomes a real limitation as the state space grows. Third, density-aware and redundancy-aware heuristics are strong practical approximations because they retain most of the value at much lower cost. Fourth, value-only greedy can fail badly, even though it sounds intuitive, because it ignores weight. Fifth, selection and compression should not be conflated: TokenPack’s strongest empirical evidence comes from solving the retrieval-time budget allocation problem before any prompt-level token pruning happens. Sixth, the LongBench v2 pilot suggests that this selection-first framing can transfer from evidence-retention proxies to downstream answering, but with an important qualification: TokenPack alone preserves near-full-context accuracy at 50% savings, whereas LongLLMLingua is slightly higher in aggregate accuracy at a similar budget. The clearer positive result is the cascade, where TokenPack-compression reaches much larger savings while keeping accuracy close to the full-context reference.

For LLM context selection, these observations imply that TokenPack should not run DP over every chunk in a huge corpus. Instead, the retrieval system should first create a candidate pool using embeddings, then apply DP or a strong approximation inside that smaller pool. The QASPER selector results also clarify what TokenPack is and is not proving at this stage. The current paper does not show that knapsack is the universally best retrieval rule under evidence recall. Rather, it shows that budgeted selection is the right abstraction, that solver quality and score quality are separable, and that a stronger score

model is now the main limiting factor. The compression and LongBench comparisons refine the claim further: TokenPack does not make the language model itself smarter, but it selects a better evidence substrate and can improve the operating point of a downstream compressor. Prompt compression can still be valuable after selection, especially when a system must push to higher savings, yet the QASPER frontier and the LongBench cascade both suggest that selection-first pipelines are a better framing than treating compression as the only budget-control step.

IX. LIMITATIONS AND FUTURE WORK

The current results are strongest on synthetic knapsack instances, evidence-oriented retrieval tests, the LLMLingua-2 comparison, and an 83-case LongBench v2 pilot. QASPER measures retention proxies rather than final human-judged answer quality, and the LongBench accuracy differences are descriptive rather than statistically significant at this sample size. The compression comparison focuses on one family of methods, uses LongLLMLingua context-level filtering rather than the full token-level compressor, and reports realized savings but not full per-document rate variance. BudgetMem is discussed as related work, but no direct numerical BudgetMem baseline is included because the study did not have an official implementation or exact reproduction artifact; the artifact includes only a BudgetMem-style feature proxy, which trails evidence-hybrid on QASPER. A larger study should add full LongLLMLingua token-level compression, a true BudgetMem reproduction if code becomes available, KV-compression baselines, more generator models, human review of disagreement cases, and learned or tuned scoring and chunk-size policies.

X. CONCLUSION

This paper models LLM context selection as a 0/1 Knapsack Problem and evaluates exact and heuristic algorithms under realistic token budgets. Exact DP is optimal but expensive, while density-aware approximations are fast and competitive. The retrieval results are intentionally nuanced: budget-top-k remains the strongest QASPER evidence-recall baseline under the current scoring function, so the paper does not claim universal knapsack dominance. Instead, the main systems finding is that retrieval-time selection and prompt compression are not interchangeable. TokenPack preserves substantially more QASPER evidence than LLMLingua-2 at matched 50% token savings, and the TokenPack-then-LongLLMLingua cascade reaches 74.5% LongBench context saving with only a small accuracy difference from full context. Future gains will depend on utility models better aligned with downstream answer quality.

REFERENCES

- [1] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*. Springer, 1972, pp. 85–103.
- [2] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.
- [3] V. Cacchiani, M. Iori, A. Locatelli, and S. Martello, “Knapsack problems: An overview of recent advances. part ii: Multiple, multidimensional, and quadratic knapsack problems,” *Computers & Operations Research*, 2022.

- [4] T. Lust and J. Teghem, "The multiobjective multidimensional knapsack problem: A survey and a new approach," *International Transactions in Operational Research*, 2010.
- [5] T. Szkaliczki, "Solution methods for the multiple-choice knapsack problem and their applications," *Mathematics*, 2025.
- [6] S. Verma, M. Pant, and V. Snasel, "A comprehensive review on nsga-ii for multi-objective combinatorial optimization problems," *IEEE Access*, 2021.
- [7] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, 2020.
- [8] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. Gonzalez, "Mengpt: Towards llms as operating systems," 2023.
- [9] N. Ratner, Y. Levine, Y. Belinkov, O. Ram, I. Magar, O. Abend, E. Karpas, A. Shashua, K. Leyton-Brown, and Y. Shoham, "Parallel context windows for large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- [10] Y. Ding, L. Zhang, C. Zhang, Y. Xu, N. Shang, J. Xu, F. Yang, and M. Yang, "Longrope: Extending llm context window beyond 2 million tokens," 2024.
- [11] D. Zhu, N. Yang, L. Wang, Y. Song, W. Wu, F. Wei, and S. Li, "Pose: Efficient context window extension of llms via positional skip-wise training," 2023.
- [12] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," 2023.
- [13] P. Xu, W. Ping, X. Wu, L. C. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoenybi, and B. Catanzaro, "Retrieval meets long context large language models," 2023.
- [14] X. Li, Y. Cao, Y. Ma, and A. Sun, "Long context vs. rag for llms: An evaluation and revisits," 2024.
- [15] N. Paulsen, "Context is what you need: The maximum effective context window for real world limits of llms," 2025.
- [16] A. Ahmed, S. Di, F. Cappello, Z. Liu, J. Han, and A. Anwar, "Systematic evaluation of optimization techniques for long-context language models," 2025.
- [17] M. Gunther, I. Mohr, D. J. Williams, B. Wang, and H. Xiao, "Late chunking: Contextual chunk embeddings using long-context embedding models," 2024.
- [18] P. Verma, "S2 chunking: A hybrid framework for document segmentation through integrated spatial and semantic analysis," 2025.
- [19] K. Luo, Z. Liu, S. Xiao, and K. Liu, "Landmark embedding: A chunking-free embedding method for retrieval augmented long-context large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [20] —, "Bge landmark embedding: A chunking-free embedding method for retrieval augmented long-context large language models," 2024.
- [21] M. A. Shaikat, M. Adnan, and C. C. N. Kuhn, "A systematic investigation of document chunking strategies and embedding sensitivity," *arXiv preprint arXiv:2603.06976*, 2026.
- [22] Y. Fang, T. Sun, Y. Shi, and X. Gu, "Attentionrag: Attention-guided context pruning in retrieval-augmented generation," *arXiv preprint arXiv:2503.10720*, 2025.
- [23] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference," 2024.
- [24] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. F. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," 2024.
- [25] Y. Zuo, S. Wei, C. Zhang, Z. Liu, W. Lu, and D. Song, "Windowkv: Task-adaptive group-wise kv cache window selection for efficient llm inference," 2025.
- [26] C. V. K. Alla, H. N. Gaddam, and M. Kommi, "Budgetmem: Learning selective memory policies for cost-efficient long-context processing in language models," *arXiv preprint arXiv:2511.04919*, 2025.
- [27] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "Llmlingua: Compressing prompts for accelerated inference of large language models," 2023.
- [28] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [29] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, "Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," 2024.
- [30] T. Hirao, Y. Yoshida, M. Nishino, N. Yasuda, and M. Nagata, "Single-document summarization as a tree knapsack problem," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, 2013.
- [31] J. Pilault, R. Li, S. Subramanian, and C. Pal, "Factorizing content and budget decisions in abstractive summarization of long documents," 2022.
- [32] A. Duchnowski, E. Pavlick, and A. Koller, "A knapsack by any other name: Presentation impacts llm performance on np-hard problems," in *Findings of the Association for Computational Linguistics: EMNLP*, 2025.
- [33] Z. Li, C. Chen, T. Yang, T. Ding, R. Sun, G. Zhang, W. Huang, and Z. Luo, "Knapsack rl: Unlocking exploration of llms via optimizing budget allocation," 2025.
- [34] S.-H. Cha, G. Kim, D. Han, T. Yang, and I. Han, "Knapspec: Self-speculative decoding via adaptive layer selection as a knapsack problem," 2026.
- [35] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks," *arXiv preprint arXiv:2412.15204*, 2024.
- [36] Qwen Team, "Qwen2.5: A party of foundation models," <https://qwenlm.github.io/blog/qwen2.5/>, 2024.
- [37] OpenAI, "Api pricing," <https://openai.com/api/pricing/>, 2026, accessed: 2026-05-05.